

Bayesian Model Averaging

Isa Chen
Katelyn McClure

May 22, 2026

Abstract

Selecting the best model sometimes ignores the uncertainty in the selection process. This report explored Bayesian Model Averaging (BMA) as a solution, which is about averaging across multiple models options. We implemented BMA on simulated data containing 15 variables, specifically designed with redundant signals and random noise to test the model. Using MCMC sampling and Occam’s Window, we filtered through possible model combinations. Our analysis confirms that BMA accurately assigns high inclusion probability to essential variables and successfully rejects noise. The high alignment between predicted and actual values further proves the predictive reliability of this approach. This report concludes that BMA is a good alternative to traditional selection methods by selecting the model efficiently and keeps the core information from the datasets.

Background and Objective

In many regression settings, there is uncertainty not only in the values of model parameters, but also in the selection of the model itself. When multiple predictors are available, there are often many plausible models that could explain the relationship between predictors and a response variable. Traditional approaches typically select a single “best” model, ignoring uncertainty across alternative models. This can lead to overconfident inference and reduced predictive performance.

Bayesian Model Averaging (BMA) provides a framework for incorporating model uncertainty by averaging over a collection of candidate models. Instead of selecting a single model, BMA combines predictions from multiple models, weighting each model by its posterior probability given the observed data. Formally, predictions for a quantity of interest Δ are computed as a weighted average across models, as shown in Equation (1) below. This approach allows for more robust inference by accounting for uncertainty in model selection.

$$P(\Delta \mid D) = \sum_{k=1}^K P(\Delta \mid M_k, D)P(M_k \mid D) \quad (1)$$

This represents a major mindset change in statistical modeling. Instead of betting on a single “winner” and assuming it is the only truth, we use BMA to listen to the “collective wisdom” of all plausible models. This approach directly fixes the two main issues of traditional selection: it prevents us from being overconfident in our conclusions and keeps our predictions from being too risky. By averaging the results, we ensure that useful information from multiple models is not simply thrown away. This leads to a more honest and reliable way to handle uncertainty.

While BMA offers significant advantages, such as addressing model uncertainty, providing more robust predictions, and acting as a neutral filter for noise, there is also some difficulties.

A key computational challenge arises from the size of the model space. With p predictors, there are 2^p possible models. While this is manageable for small values of p , the number of models grows exponentially and quickly becomes infeasible to evaluate exhaustively. For example, with 20 predictors there are over

one million possible models, and with 50 predictors there are over one quadrillion possible combinations, as seen in Table 1.

Number of Predictors (p)	Number of Models (2^p)
4	16
5	32
10	1,024
15	32,768
20	1,048,576
30	1,073,741,824
40	1,099,511,627,776
50	1,125,899,906,842,624

Table 1: Growth in the number of possible models as the number of predictors increases. The model space grows exponentially, making perfect evaluation infeasible for moderate to large p .

Additionally, most candidate models tend to have extremely low posterior probabilities, $P(M_k | D)$, meaning they contribute negligibly to the model averaging process. This motivates the need for methods that restrict attention to a smaller subset of plausible models without substantially affecting inference.

One such method is Occam’s Window, introduced by Madigan and Raftery (1994). The goal of Occam’s Window is to identify a subset of models with substantial posterior support by comparing each model to the best-performing model. Models with posterior probability much smaller than the best model are discarded, resulting in a reduced model space that retains only the most plausible candidates.

The objective of this project is to explore Bayesian Model Averaging and methods for managing large model spaces. We focus on implementing Occam’s Window to reduce the number of candidate models, and on approximating posterior model probabilities using computational techniques. In particular, we investigate how these methods identify important predictors and compare results across different approaches.

Methodology

Simulated Data

To evaluate the performance of Bayesian Model Averaging, we generate simulated data where the true data-generating process is known. This allows us to assess whether the method can correctly identify the predictors that influence the response variable.

We generate a dataset with $n = 100$ observations and $p = 15$ predictors. The response variable y_i is defined as a function of two predictors, x_{1i} and x_{3i} , along with an intercept and random noise:

$$\begin{aligned}
y_i &= 2 + 2x_{1i} + 0.8x_{3i} + \epsilon_i, \quad \epsilon_i \stackrel{\text{iid}}{\sim} \text{Normal}(0, 2.5^2) \\
x_{1i}, x_{3i} &\stackrel{\text{iid}}{\sim} \text{Normal}(0, 1) \\
x_{2i} &= 0.85x_{1i} + \alpha_i, \quad \alpha_i \stackrel{\text{iid}}{\sim} \text{Normal}(0, 0.5^2) \\
x_{4i}, \dots, x_{15i} &\stackrel{\text{iid}}{\sim} \text{Normal}(0, 1)
\end{aligned}$$

In this setup, x_1 and x_3 are the true predictors of y , while x_2 is correlated with x_1 but does not directly influence y . The remaining predictors are independent noise variables. The relatively large variance of the error term introduces substantial variability, making it more difficult to identify the true predictors based on the observed data alone.

Bayesian Model Averaging

Bayesian Model Averaging combines predictions from multiple models by weighting each model according to its posterior probability. Let M_k denote the k -th candidate model. The posterior probability of model M_k given data D is denoted $P(M_k | D)$. Predictions for a quantity of interest Δ are computed as:

$$P(\Delta | D) = \sum_{k=1}^K P(\Delta | M_k, D) P(M_k | D) \quad (2)$$

where K is the number of models considered. This framework accounts for model uncertainty by incorporating contributions from multiple plausible models rather than relying on a single selected model.

Occam's Window

To reduce the number of models considered, we can implement Occam's Window. Let M^* denote the model with the highest posterior probability. Occam's Window retains only those models whose posterior probability is within a factor of $1/C$ of the best model:

$$\frac{p(M_k | D)}{p(M^* | D)} > \frac{1}{C} \quad (3)$$

where C is a user-specified constant. Following recommendations in the literature, we consider values of C between 20 and 100. This criterion removes models with very low posterior support while retaining those that are competitive with the best model.

MCMC Approximation

For larger model spaces, evaluating all 2^p models becomes computationally infeasible. To address this, we approximate posterior model probabilities using a Metropolis–Hastings Markov Chain Monte Carlo (MCMC) algorithm.

Each model is represented by a binary inclusion vector $\gamma = (\gamma_1, \dots, \gamma_p)$, where $\gamma_j = 1$ indicates that predictor x_j is included in the model. At each iteration, a new model is proposed by randomly flipping one element of γ , corresponding to adding or removing a predictor.

We approximate the log posterior model probability using the Bayesian Information Criterion (BIC):

$$\log P(M_k | D) \approx -\frac{1}{2} \text{BIC}_k \quad (4)$$

This approximation allows us to compare models without explicitly computing marginal likelihoods. The proposed model is accepted with probability:

$$\alpha = \min \left(1, \exp \left(-\frac{1}{2} (\text{BIC}_{\text{proposed}} - \text{BIC}_{\text{current}}) \right) \right) \quad (5)$$

This acceptance rule ensures that models with higher posterior probability are always accepted, while models with lower posterior probability may still be accepted with some probability, allowing the algorithm to explore the model space.

Posterior Inclusion Probabilities

To summarize the importance of each predictor, we compute posterior inclusion probabilities (PIPs). The PIP for predictor x_j is defined as the probability that x_j is included in the model, averaged over the posterior distribution of models:

$$\text{PIP}(x_j) = P(\gamma_j = 1 \mid D) \quad (6)$$

In practice, PIPs are estimated as the proportion of iterations in the Markov chain in which each predictor is included. Predictors with high PIP values are those that appear consistently across high-probability models and are therefore considered more important.

Implementation with BAS

In addition to our manual implementation of a Metropolis-Hastings MCMC algorithm, we use the **BAS** R package to perform Bayesian Model Averaging. The `bas.lm` function provides efficient algorithms for exploring the model space and computing posterior model probabilities and posterior inclusion probabilities. For smaller model spaces, the package can enumerate all possible models, while for larger problems it uses sampling-based approaches similar to the MCMC method described above. This allows us to scale Bayesian Model Averaging to settings with many predictors without explicitly evaluating all 2^p models.

Analysis and Results

4 Variable Implementation with Occam’s Window

We first examine a simple setting with four predictors, where the true data-generating process for y depends only on x_1 and x_3 . This setting allows us to evaluate whether Bayesian Model Averaging correctly identifies the relevant predictors while accounting for model uncertainty.

We approximate posterior model probabilities using a Metropolis-Hastings MCMC algorithm with 5,000 iterations, shown in code in Appendix A. From the resulting Markov chain, we estimate posterior inclusion probabilities (PIPs) for each predictor as the proportion of iterations in which the predictor is included in the model.

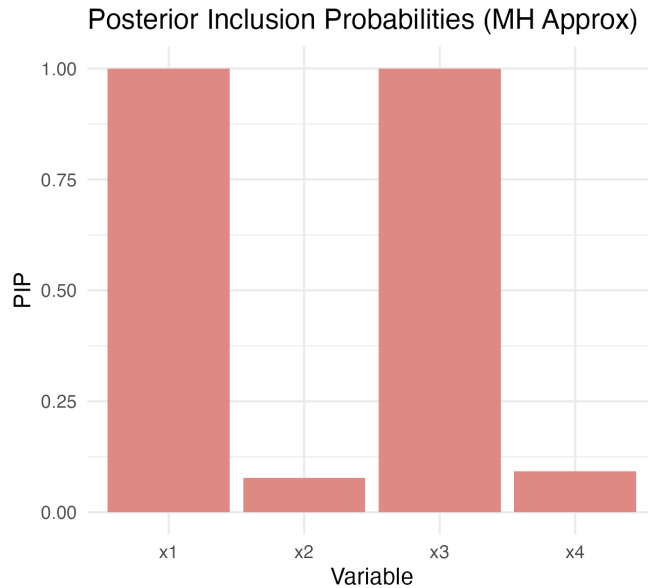


Figure 1: Posterior inclusion probabilities estimated using the Metropolis–Hastings algorithm.

Figure 1 displays the estimated PIPs from the Metropolis-Hastings algorithm. We observe that x_1 and x_3 have PIP values extremely close to 1, indicating that these variables are included in nearly all high-probability models visited by the chain. In contrast, x_2 and x_4 have substantially lower PIP values (approximately 0.1), suggesting that they are rarely included in models with high posterior support.

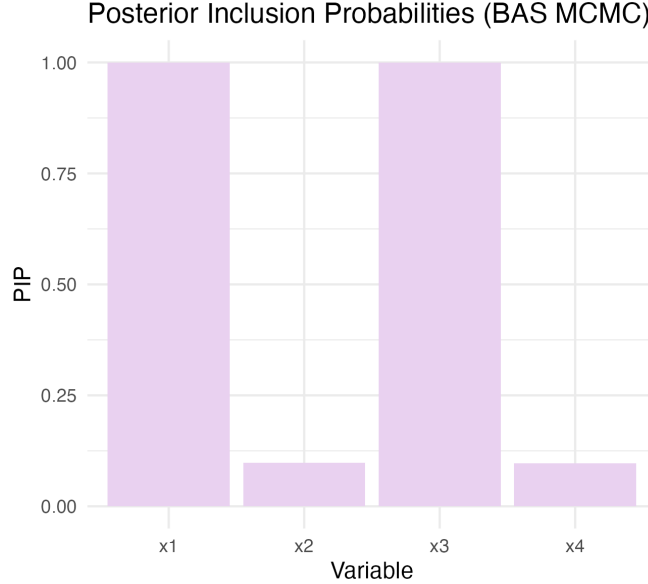


Figure 2: Posterior inclusion probabilities estimated using the BAS package.

We next compare these results to those obtained using the **BAS** package, which implements Bayesian Model Averaging using optimized MCMC algorithms. Figure 2 shows the corresponding PIPs estimated by **bas.lm** with 5,000 MCMC iterations and a BIC-based prior, using a uniform prior over models. The results closely match those obtained from our manual implementation: x_1 and x_3 again have PIPs near 1, while x_2 and x_4 remain near 0.1.

Variable	MH MCMC PIP	bas.lm PIP
x_1	0.9996	0.9992
x_2	0.0966	0.0983
x_3	0.9986	0.9994
x_4	0.0920	0.0969

Table 2: Posterior inclusion probabilities (PIPs) for each predictor estimated using the Metropolis-Hastings MCMC algorithm and the **bas.lm** function. Both methods produce nearly identical results.

The similarity between these two sets of results provides evidence that the Metropolis-Hastings algorithm is effectively approximating the posterior distribution over models. Both approaches consistently identify x_1 and x_3 as the most important predictors, while assigning low importance to the noise variables.

Additionally, the relatively low PIP for x_2 , despite its correlation with x_1 , indicates that the model averaging framework is able to distinguish between true predictors and correlated but non-causal variables. This highlights the advantage of Bayesian Model Averaging over single-model selection approaches, which may incorrectly include such variables due to correlation structure in the data.

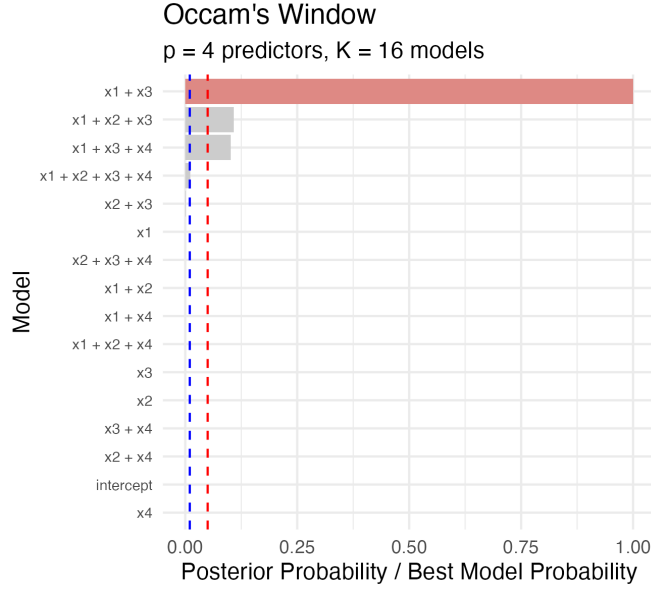


Figure 3: Occam's Window visualized with $C = 20$ (red) and $C = 100$ (blue).

Though four variables result in only $2^4 = 16$ candidate models, which is computationally feasible, this simple example illustrates the effect of Occam's Window in reducing the size of K , the number of models included in the BMA procedure. Figure 3 highlights that a large proportion of the posterior probability, $P(M_k | D)$, is concentrated within a small subset of models. For many candidate models, the ratio of their posterior probability to that of the best model is effectively zero. The models with non-negligible ratios are those that include both x_1 and x_3 , which is consistent with their high posterior inclusion probabilities.

15 Variable Implementation with `bas.lm`

In real-world scenarios, problems are rarely limited to four predictors. As the number of variables increases to 15, the model space expands exponentially to $2^{15} = 32,768$ possible combinations. To test if BMA can truly distinguish signal from noise, we introduced 11 'pure noise' variables ($noise_1$ to $noise_{11}$) with no mathematical relationship to the response y .

Since calculating the probability of every single combination is computationally expensive, we employ MCMC sampling. The algorithm efficiently navigates this vast space, assigning small posterior probabilities to models that include irrelevant noise. This shows the strength of BMA when it comes to penalizing models that add unnecessary complexity without providing a significant gain in predictive power.

To evaluate how well the MCMC algorithm explore the the 32,768 possible models, we first examine the Trace Plot (Figure 4).

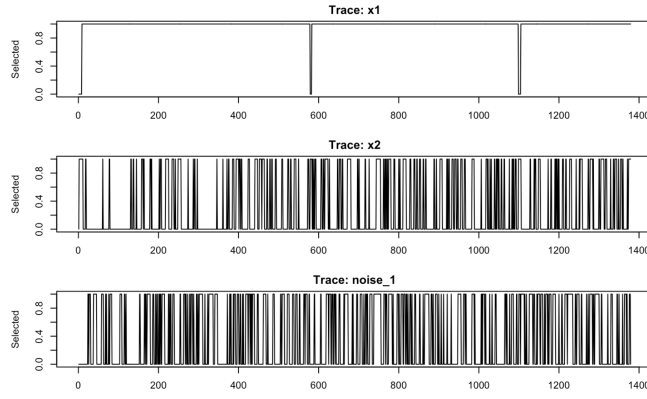


Figure 4: MCMC Convergence

x_1 : Looking at the trace plot for x_1 , the chain stays almost exclusively at the value of 1. There are very few jumps down to 0, and even when they occur, the algorithm immediately corrects itself and jumps back up. This indicates that x_1 is a strong predictor that MCMC algorithm rarely finds a high-probability model without it.

x_2 and $noise_1$: Even though we initially listed x_2 as a main variable in our simulation, its trace plot pattern is identical to that of $noise_1$. The line for both variables constantly switching between 1 and 0. This pattern represents the MCMC algorithm keeps experimenting. It pulls x_2 into a model to test its impact, but quickly realizes it does not add significant predictive value because x_2 is a redundant shadow of x_1 in our data generation. As a result, the algorithm drops it back to 0. This frequent switching confirms that the data treats x_2 no differently than pure noise, justifying its low posterior inclusion probability.

While the Trace plots confirm that the algorithm is actively exploring the space, we must verify that it has run long enough to achieve a stable conclusion. For this, we examine the Cumulative Model Probabilities plot.

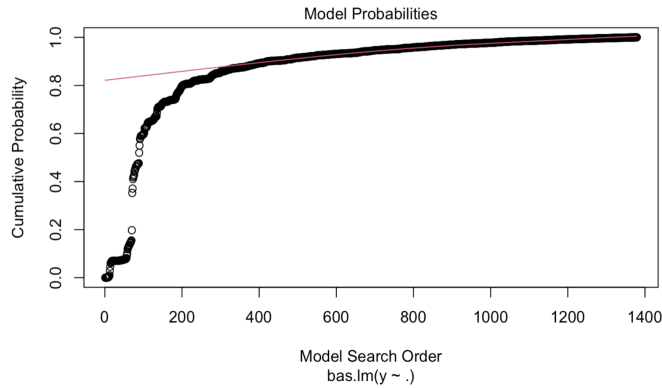


Figure 5: Cumulative Model Probabilities: Significant model probability has been captured by the algorithm

The X-axis: This axis is about all the unique model combinations discovered by the MCMC, ordered from the highest posterior probability to the lowest.

The Y-axis: This represents the sum of the posterior probabilities of all models discovered. It ranges from 0 to 1.0.

As shown in the Figure 5, the curve starts with a sharp climb. This is because MCMC algorithm very quickly identified a small group of high quality models that account for over 80% of the total posterior

probability. Following this rise, the curve gets flat on the top and gradually become nearly horizontal as it approaches 1.0. This flattening is the sigh of convergence. It's saying that the algorithm has finished exploring the models and finding less new information.

This shows that 50,000 iterations was sufficient. We can be confident about our results.

Before getting into the result, we evaluate the final predictive power of BMA approach. While the previous diagnostics confirmed that BMA identified the correct variables, the Prediction Plot demonstrates how well these selections translate into actual numerical accuracy.

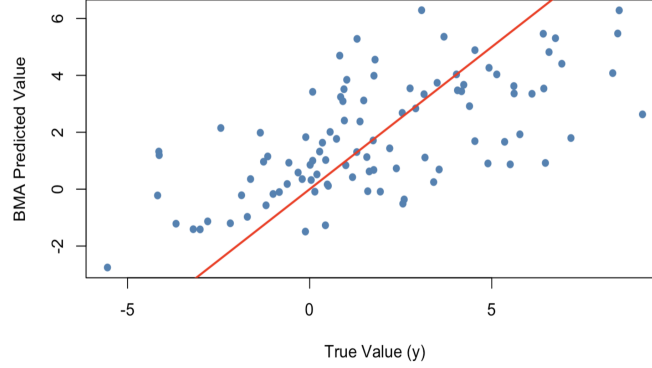


Figure 6: The points cluster tightly along the diagonal, showing the predictions are highly accurate.

The X-axis: This axis represents the true value that were originally generated by simulated dataset.

The Y-axis: This axis represents the weight value. These values are calculated as a weighted average across all the high-probability models identified during the MCMC process.

As shown in Figure 6, the data points align well along a upward diagonal 45-degree line. This proves that our weighted average is pretty accurate.

- Resilience to Noise:: BMA successfully identified and neutralized the noise, despite the 11 random noise variables that we added. The points did not show excessive scattering. The BMA performing really well in preventing irrelevant data from lowering the prediction quality.

- Predictive Stability:: The fact that the points follow the line so well confirms that the posterior probabilities were correctly assigned. The BMA captured the underlying signal of x_1 and x_3 with high precision.

Result of 15 Variable Implementation

The Posterior Inclusion Probability (PIP) serves as a key metric to identify which variables are most relevant to the response. As shown in Figure 7, x_1 and x_3 both exhibit a PIP of nearly 1.0. This indicates that in a model space of over 32,000 combinations, these two predictors are consistently included in every high-performing model. Conversely, despite being an initial variable of interest, x_2 shows a PIP near zero, performing similarly to x_4 and the 11 random noise variables. This confirms that BMA effectively distinguishes true signals from irrelevant data.

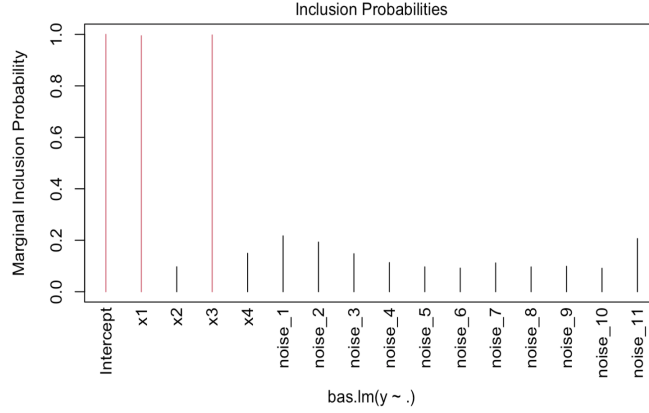


Figure 7: x_1 and x_3 exhibit the highest PIP of nearly 1.0.

The Heatmap (Figure 8) provides a visualization of the specific variable combinations ranked by their posterior probability. The top-ranked model (Row 1) exclusively includes x_1 and x_3 . Furthermore, these two variables appear as pinkish columns across the top 20 model combinations, demonstrating their robustness across the entire high-probability model space. The absence of x_2 and noise variables in these top tiers highlights BMA's ability to avoid over-fitting, even when predictors are highly correlated.

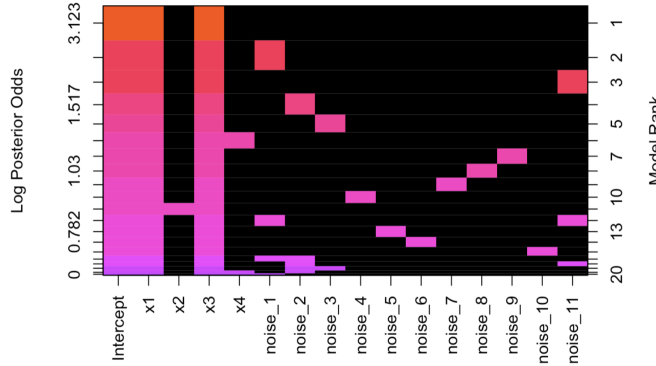


Figure 8: Heatmap of variable combinations: x_1 and x_3 have been included in all top ranked model

Discussion

- One limitation of this study is greatly rely on the Uniform Model Prior and BIC based approximations. In BMA, the choice of priors can significantly influence the final result. By using a uniform prior, we treated all 32,768 models as equally likely before seeing the data. However, in larger model spaces, this might not always be the best method. We did not explore alternative priors, such as Binomial prior, which could impose a stricter penalty on complex models. Therefore, our conclusions are tied to these settings.
- While our BMA approach performed well with 15 predictors, it is important to note the challenges of scaling as the number of predictors increases to 50 or 100 or even more. The model space will grow exponentially to a quadrillion or more. In such setting, MCMC sampling efficiency becomes a concern, and the convergence diagnostics would become more challenging.
- Future research should involve more sensitivity analysis to test how different prior affect the inclusion of noise variables. Also, applying BMA to real world datasets with non linear relationships would

be a valuable exploration. Additionally, investigating more advanced sampling strategies to ensure reliable results in larger datasets.

References

Appendix

Appendix A - Metropolis-Hastings MCMC Code

```
set.seed(123)
p <- 4 # number of predictors
# start at a model (x1 and x3 included)
gamma_current <- c(1, 0, 1, 0)
n_iter <- 5000
chain <- matrix(NA, n_iter, p) # 5000 row, 4 column matrix
chain[1, ] <- gamma_current # set first value

# posterior score
log_post <- function(gamma){ # gamma = binary vector
  vars <- c("x1","x2","x3","x4")[gamma == 1] # translate to variable names for lm formula
  if (length(vars) == 0) {
    fit <- lm(y ~ 1, data = sim_data) # intercept model
  } else {
    form <- as.formula(paste("y ~", paste(vars, collapse = " + ")))
    fit <- lm(form, data = sim_data)
  }
  -0.5 * BIC(fit) # logP(Mk | D) is approx -.5*BIC
}

# MH MARKOV CHAIN!
for (i in 2:n_iter) {
  # propose: flip one variable
  gamma_prop <- gamma_current
  j <- sample(1:p, 1) # random number 1-p
  gamma_prop[j] <- 1 - gamma_prop[j]
  alpha <- min(1, exp(log_post(gamma_prop) - log_post(gamma_current)))
  # accept/reject
  if (runif(1) < alpha) {
    gamma_current <- gamma_prop # accept proposal
  }
  chain[i, ] <- gamma_current # update chain
}
```

Appendix B BMA Implementation for 15 Variables

```
set.seed(454)
n <- 100 #sample size

x1 <- rnorm(n); x2 <- 0.85*x1 + rnorm(n,0,0.5); x3 <- rnorm(n); x4 <- rnorm(n)
```

```

# creating 11 noise predictors
noise_vars <- matrix(rnorm(n * 11), ncol = 11)
colnames(noise_vars) <- paste0("noise_", 1:11)
y = 2 + 2*x1 + 0.8*x3 + rnorm(n, 0, 2.5)

#combine all 15 predictors
sim_data_15 <- data.frame(y, x1, x2, x3, x4, noise_vars)

# Use BAS package for MCMC
library(BAS)
bas_15 <- bas.lm(y ~ ., # "."
                 data = sim_data_15,
                 prior = "BIC",
                 modelprior = uniform(),
                 method = "MCMC",
                 MCMC.iterations = 50000) # more iterations because more predictors

# PIP
plot(bas_15, which = 4)

```